

Adaptive Meta-Cognitive Neuro-Symbolic AI for Urban Traffic Flow Prediction in Edge-Enabled Transportation Systems

Devesh Vaidya, CO24317
co24317@ccet.ac.in

Abstract—Urbanization along with high vehicle density has led to increased traffic congestion, causing economic, environmental, and transportation inefficiencies. Conventional traffic forecasting systems generally find it difficult to deal with spatio-temporal correlations and lack the property of interpretability and adaptiveness in traffic environments. The project aims to build an Adaptive Meta-Cognitive Neuro-Symbolic AI Framework for Urban Traffic Flow Prediction in Edge-Enabled Transportation Systems, which leverages deep learning, symbolic reasoning, and self-evaluation processes to deliver accurate, explainable, and adaptive traffic flow predictions. The proposed system utilizes a Graph WaveNet-based Spatio-Temporal Graph Neural Network (ST-GNN) to identify spatial relationships between traffic sensors and traffic patterns over time using benchmark datasets like METR-LA. An augmented Neuro-Symbolic reasoning engine enhances the learning process with domain knowledge such as traffic rules, weather data, incidents, and congestion spread. In addition, an Adaptive Meta-Cognitive Evaluation unit continuously monitors prediction accuracy, detects novel traffic instances, checks consistency in traffic rules, and modifies the prediction strategy. This project focuses on creating an Intelligent Transportation System in real time by implementing the proposed system in an edge-enabled architecture. For implementation of intelligent transportation systems that provide real-time functionality, the proposed architecture is intended to be deployed in an environment where edge computing has already been established. Results from experiments using popular forecasting metrics such as MAE, RMSE, and MAPE demonstrate the superiority of the proposed architecture over traditional deep learning algorithms.

Index Terms—Adaptive Meta-Cognitive AI, Edge-Enabled Transportation Systems, Spatio-Temporal Graph Neural Networks, Neuro-Symbolic AI, Explainable Artificial Intelligence, Edge Computing.

I. INTRODUCTION

A. Overview

In the past decades, there have been tremendous changes in transportation systems due to advance-

ments in sensing technology, communication networks, and computational intelligence [1]. Conventional transportation management systems have now been transformed into Intelligent Transportation Systems (ITS), whereby information and communication technologies are embedded in transportation infrastructure to enhance efficiency, sustainability, safety, and mobility. Modern ITS environments depend heavily on traffic sensors [2], edge computing [3], and artificial intelligence algorithms to collect and analyze vast amounts of traffic data in real time. In the early days of transportation management, most systems relied on static controls and historical traffic data. This approach was effective to a degree, but it could not respond well to ever-changing traffic dynamics. With current technologies, transportation systems have become dynamic and predictive, requiring advanced computational algorithms capable of handling complex spatiotemporal traffic dynamics. Meanwhile, rapid urbanization has introduced new challenges for cities worldwide. Growing populations combined with the expansion of metropolitan areas have resulted in a considerable rise in the number of vehicles using road networks.

B. Need for Accurate Traffic Flow Prediction

The existing transportation infrastructure is often overwhelmed, resulting in serious traffic congestion, longer commute times, high fuel consumption, and increased greenhouse gas emissions. The problems of congestion affect more than travel time; they also carry economic consequences due to lower productivity and higher transportation costs [4], and they reduce the overall quality of life in cities. Traditional approaches to traffic control often fail to cope with such dynamic and nonlinear traffic systems, motivating the development of intelligent ap-

proaches capable of handling traffic dynamics. Traffic flow prediction is one of the essential elements of ITS. Reliable traffic prediction allows transportation organizations to forecast congestion, optimize traffic signal control policies, provide intelligent routing solutions, increase transportation efficiency, and manage emergencies. These systems exhibit high spatial and temporal dependency, and accurate prediction is possible only through computational techniques that can learn from traffic patterns and adapt to changing environmental conditions.

C. Problem Statement

Traffic prediction is a challenging problem due to the dynamic, nonlinear, and uncertain nature of urban traffic environments. Traditional traffic forecasting approaches, including statistical models such as ARIMA [5] and machine learning algorithms such as Support Vector Regression and Random Forests, are unable to effectively capture the complex spatio-temporal dependencies present in large-scale traffic networks. Their predictive performance deteriorates considerably when traffic patterns become highly dynamic or when unexpected incidents occur. State-of-the-art deep learning approaches, particularly Graph Neural Networks [6] and recurrent architectures, often function as black-box systems. While they produce accurate predictions, they fail to provide human-understandable explanations regarding:

- 1) Why congestion is expected to occur.
- 2) Which factors influenced the prediction.
- 3) How external conditions affected the decision-making process.

The absence of explainability reduces user trust and limits the adoption of AI-based traffic management systems in safety-critical transportation environments. Most existing forecasting models assume relatively stable traffic conditions and exhibit poor generalization when encountering previously unseen scenarios or anomalous traffic patterns. Current traffic prediction systems rarely possess the ability to assess the reliability of their own predictions or adapt their reasoning strategies when uncertainty increases. There is a need for intelligent systems capable of:

- (i) Measuring prediction confidence,
- (ii) Detecting novel traffic situations,
- (iii) Identifying inconsistencies in reasoning,

- (iv) Performing self-correction and adaptive learning.

Therefore, there exists a significant research gap in developing an integrated framework that combines high predictive performance, symbolic reasoning, explainability, and meta-cognitive self-evaluation for urban traffic prediction.

D. Research Objectives

The main purpose of this study is to design and develop an Adaptive Meta-Cognitive Neuro-Symbolic Artificial Intelligence system for producing precise, interpretable, and adaptive traffic flow predictions in edge-enabled ITS [3].

- 1) **Objective 1 – Design a Neuro-Symbolic AI System:** To create a hybrid architecture based on ST-GNN and symbolic reasoning systems for modeling data-driven and domain-specific traffic knowledge.
- 2) **Objective 2 – Integrate Adaptive Meta-Cognitive Assessment:** To create a meta-cognitive system that constantly assesses prediction certainty, recognizes novel traffic situations, evaluates rule consistency, and adjusts the prediction system through adaptive learning and self-correction processes.
- 3) **Objective 3 – Generate Interpretable Traffic Predictions:** To generate interpretable and human-understandable traffic predictions in the form of symbolic explanations, certainty assessment, and causal reasoning.
- 4) **Objective 4 – Support Deployment on Edge-Enabled Infrastructure:** To design and develop the proposed system for implementation in edge computing environments, enabling low-latency traffic forecasting and distributed computing.
- 5) **Objective 5 – Increase Prediction Accuracy and Reliability:** To improve the efficiency and performance of traffic prediction systems through the use of deep learning, symbolic reasoning, and adaptive meta-cognition techniques.

II. URBAN TRAFFIC PREDICTION SYSTEMS

Urban transportation systems have become highly complex networks. Rapid urbanization, the increasing number of vehicles, and the expansion of metropolitan areas have greatly aggravated traffic

congestion around the world. Therefore, designing precise and intelligent traffic prediction systems has become one of the main research problems within the ITS field. Traffic prediction systems forecast conditions such as speed, volume, density, and congestion using historical and current data acquired from transportation infrastructure. Modern ITS collects data from loop detectors, surveillance video cameras [7], GPS-equipped vehicles, weather stations, and incident management systems. After processing, the resulting predictions can be used for route planning, congestion management, traffic signal control, and emergency response. Traffic data exhibit a high level of temporal dependency, since the present traffic condition depends heavily on historical traffic. Traffic data also exhibit high spatial dependency, because congestion on one road segment affects traffic in adjacent areas. Moreover, traffic data are highly nonlinear and prone to sudden changes caused by accidents, roadblocks, adverse weather, and other public events.

A. Machine Learning Approaches

Before the development of deep learning techniques, traffic prediction was mostly based on traditional machine learning approaches that learned the relationship between past traffic data and future traffic states through manually designed features and statistical techniques.

- 1) **Linear Regression (LR)** was one of the first methods used, in which the future traffic state was considered a linear combination of past traffic observations and explanatory variables. This approach is simple and efficient but lacks the capability to model more complex traffic patterns [8].
- 2) **Support Vector Regression (SVR)** generalizes the Support Vector Machine approach for regression problems, finding a function that approximately fits observations within a defined error margin. Using kernel functions such as the radial basis and polynomial kernels, SVR can model nonlinear interactions and generally outperforms linear models for traffic prediction.
- 3) **Random Forest (RF)** is an ensemble technique that produces predictions by averaging outputs from a set of randomly constructed decision trees obtained via bootstrapping and

random feature selection. This approach can successfully model nonlinear interactions and incorporate diverse traffic-related features such as weather data, time features, and sensor data. However, Random Forest treats past observations independently and does not account for temporal and spatial relations between neighboring traffic sensors [9].

B. Deep Learning Approaches

The limitations associated with conventional machine learning approaches have driven the adoption of deep learning methods, which allow automatic hierarchical representation learning of traffic data.

One of the earliest deep learning [10] models used for traffic prediction is the Recurrent Neural Network (RNN). Using an evolving hidden state, RNNs can handle sequential data and short-term temporal dependencies. However, RNNs suffer from vanishing and exploding gradient problems, which prevent them from modeling long-term patterns.

Memory cells and gating mechanisms, including input, forget, and output gates, allow Long Short-Term Memory (LSTM) networks to overcome these limitations, making LSTM effective for traffic data with daily and weekly periodicity. Nonetheless, LSTM handles temporal dependencies well but fails to account for spatial dependence between sensors.

Gated Recurrent Units (GRU), with fewer parameters and performance comparable to LSTMs, make real-time traffic prediction with RNN-based architectures computationally cheaper. However, despite these advantages, all RNN variants discussed lack the capability of capturing transportation network topology.

C. Spatio-Temporal Graph Neural Networks

Traffic networks can naturally be expressed as graphs, with sensors as nodes and roads as edges [3]. Spatio-Temporal Graph Neural Networks (ST-GNNs) utilize this graph structure to capture spatial and temporal dependencies simultaneously. A graph is represented as $G = (V, E, A)$, where V refers to sensors, E to connections, and A to the adjacency matrix. One of the most commonly used datasets in this field is METR-LA, which comprises 207 sensors connected by a distance-based adjacency matrix.

- (a) **STGCN** uses graph convolution and temporal convolution to detect local spatial dependencies and temporal patterns while providing efficient computation and high prediction performance [11].
- (b) **DCRNN** models traffic flow as a bidirectional diffusion process, considering both upstream and downstream dependencies, and achieves strong prediction performance for long horizons.
- (c) **Graph WaveNet** combines graph convolution with dilated causal convolution and an adaptive adjacency matrix, enabling the detection of long temporal dependencies and the discovery of hidden spatial dependencies. Visualization and clustering of node interactions further contribute to the interpretability of traffic congestion propagation.

D. Neuro-Symbolic Artificial Intelligence

Despite the strong results achieved by deep learning in traffic prediction, these algorithms remain black boxes whose decisions are hard to understand and validate. Neuro-Symbolic Artificial Intelligence addresses this issue by combining neural learning with symbolic reasoning, thereby enhancing both prediction quality and explanation. A key element of the neuro-symbolic approach is knowledge representation, which involves encoding domain knowledge in a form understandable to an intelligent system. For transportation systems, this includes traffic rules, congestion dependencies, incident relations, temporal dependencies, and geographical constraints. This knowledge is commonly encoded using knowledge graphs, ontologies, production rules, and logic predicates.

Logical reasoning further advances the system by drawing conclusions based on symbolic knowledge and justifying why certain traffic situations arise – for example, that rain has slowed traffic, that an accident has caused downstream congestion, or that reduced speeds at adjacent sensors indicate emerging congestion. To increase the interpretability of the intelligent transportation system, techniques such as prediction clustering, node-importance analysis, verbal explanation generation, influential-sensor visualization, and Monte Carlo Tree Search (MCTS) are employed [12].

E. Meta-Cognitive Artificial Intelligence

In Meta-Cognitive Artificial Intelligence, systems are advanced by allowing them to reason about and evaluate their own behavior rather than simply making predictions. Such systems monitor themselves, estimate prediction accuracy, and adapt to changes in the environment. Self-monitoring is an integral part of this capability and includes monitoring prediction errors, detecting concept drift, identifying abnormal traffic patterns, and evaluating model reliability. This capability is vital for traffic prediction given the high volatility of urban traffic and its frequent deviation from historical trends.

Another critical characteristic is confidence estimation, which represents the certainty level of a model's prediction and allows the system to detect situations in which predictions cannot be trusted. Approaches to uncertainty estimation include prediction interval estimation, ensemble-based uncertainty measures, Bayesian inference, and modeling of the error distribution. Confidence estimation provides useful information when AI is applied to safety-critical transportation problems.

F. Adaptive Learning

Meta-cognitive Artificial Intelligence takes reasoning ability a step further, allowing an AI system to analyze its own behavior rather than merely making predictions. The system monitors itself, assesses its predictions, and adapts to new situations. Self-monitoring allows prediction errors to be tracked, concept drifts to be detected, abnormal traffic behaviors to be identified, and overall model reliability to be assessed. These meta-cognitive abilities are especially useful in traffic prediction, where conditions change rapidly and do not always follow past trends. Adaptive learning allows models to adjust themselves based on new information as it becomes available. Transportation systems regularly deal with challenges such as:

- (a) Schedule changes.
- (b) Changes in infrastructure.
- (c) Road construction.
- (d) Large-scale events.
- (e) Random occurrences.

G. Edge Computing in Transportation Systems

The increasing volume of traffic data generated by current ITS infrastructure introduces significant

computational and communication challenges. A typical limitation of cloud-based [13] approaches is high latency and limited bandwidth. Edge computing mitigates these problems by bringing computation closer to the sources of traffic data, such as traffic sensors, roadside units, and intelligent vehicles. Edge-based transportation systems thus distribute computational [1] activity across traffic sensors, edge servers, roadside infrastructure, and centralized cloud systems.

For traffic prediction specifically, the edge computing paradigm allows the use of lightweight deep learning models and inference conducted directly at the edge. Combining ST-GNN and neuro-symbolic reasoning techniques makes it possible to develop accurate and efficient ITS. The integration of ST-GNN, Neuro-Symbolic AI, Meta-Cognitive AI, and edge computing can become a breakthrough for future transportation systems capable of providing accurate traffic predictions, explaining their rationale, assessing their own performance, and adapting to change.

III. SYSTEM ARCHITECTURE

The proposed framework is organized into a layered architecture consisting of a data management layer, a graph construction layer, a prediction layer, and an experimental evaluation and analysis layer, each of which is discussed in the following subsections.

A. Data Management Layer

Any smart traffic prediction system requires the efficient handling of traffic data. In the METR-LA dataset, traffic data are captured using 207 loop detectors placed across the highways of Los Angeles. Traffic speed is monitored by the sensors every five minutes, producing a massive multivariate time series consisting of over 34,000 observations. The data management module acquires, stores, synchronizes, and processes the raw time series observations to prepare them for graph representation within deep learning models.

1) *Data Storage*: Traffic data are captured in multidimensional tensors in which rows correspond to timestamps and columns correspond to sensor nodes. In the original architecture, data are stored in HDF5 format, which allows efficient compression together with fast data access. A typical loading procedure is shown in Listing 1.

Listing 1: Loading and inspecting the traffic data tensor.

```
import pandas as pd

df = pd.read_hdf("metr-la.h5", key="df")
print("Traffic data shape:", df.shape)
print("Timesteps:", df.shape[0])
print("Sensors:", df.shape[1])
```

After loading, the traffic tensor can be represented as

$$X \in \mathbb{R}^{T \times N} \quad (1)$$

where T denotes the number of timestamps and N denotes the number of sensors. For deep learning applications, additional feature dimensions are introduced, leading to

$$X \in \mathbb{R}^{T \times N \times F} \quad (2)$$

where F represents the number of features per sensor. The traffic measurements are further normalized using z -score normalization, as shown in Listing 2.

Listing 2: Feature normalization using standard scaling.

```
from sklearn.preprocessing import
    StandardScaler

scaler = StandardScaler()
data_scaled = scaler.fit_transform(df.values)
```

2) *Data Synchronization*: Strict temporal consistency is required for traffic forecasting across all sensors. Because data are gathered from geographically dispersed sensors, synchronization procedures ensure that all sensor readings correspond to the same five-minute interval, as illustrated in Listing 3.

Listing 3: Temporal synchronization of sensor data.

```
df.index = pd.to_datetime(df.index)
df = df.sort_index()
df = df.resample("5min").mean()
```

Synchronization is particularly important because graph neural networks [14] assume that every node state at time t belongs to the same temporal snapshot; any temporal misalignment can distort the spatial relationships learned by the model. The synchronized tensor therefore becomes

$$X_t = [x_1^t, x_2^t, \dots, x_N^t] \quad (3)$$

where x_i^t represents the traffic speed measured by sensor i at time t .

3) *Missing Data Handling*: Real-world traffic systems frequently suffer from missing observations due to sensor malfunctions, communication failures, maintenance activities, or delayed transmissions. The proposed model uses interpolation and forward/backward-filling techniques to address missing values, as shown in Listing 4, since graph neural networks require complete node feature matrices during both training and inference.

Listing 4: Handling missing values through interpolation and filling.

```
df = df.interpolate(method="linear")
df = df.fillna(method="ffill")
df = df.fillna(method="bfill")
```

More formally, if a sensor observation is missing, i.e., $x_i^t = \text{NaN}$, it can be approximated using linear interpolation:

$$x_i^t = \frac{x_i^{t-1} + x_i^{t+1}}{2} \quad (4)$$

B. Graph Construction Layer

Graph-structured data represent a natural data structure for traffic systems. Road networks and traffic sensors exhibit high levels of spatial dependency, meaning that congestion in one area can readily propagate to surrounding areas.

1) *Sensor Graph Creation*: Each traffic sensor is modeled as a graph node:

$$V = \{v_1, v_2, \dots, v_N\} \quad (5)$$

For the METR-LA dataset, $N = 207$. The graph is therefore defined as $G = (V, E)$, where V represents sensors and E represents the spatial relationships between them. Sensor coordinates are loaded from metadata files, and the notebooks further visualize the sensor network to illustrate the spatial distribution of traffic detectors across Los Angeles.

2) *Adjacency Matrix Construction*: The adjacency matrix indicates connectivity between nodes – in this case, between sensors – and can be converted into the edge representation required by graph neural networks:

$$A \in \mathbb{R}^{N \times N} \quad (6)$$

where

$$A_{ij} = \begin{cases} 1, & \text{if sensors } i, j \text{ are connected} \\ 0, & \text{otherwise} \end{cases} \quad (7)$$

The implementation loads the precomputed adjacency matrix and extracts the corresponding edge index, as shown in Listing 5.

Listing 5: Loading the adjacency matrix and extracting the edge index.

```
import numpy as np

adj_mx = np.load("adj_mx.npy")
print(adj_mx.shape) # (207, 207)

edge_index = np.array(np.nonzero(adj_mx))
```

This representation stores source-node-to-destination-node pairs for every graph connection.

3) *Edge Weight Generation*: The notebooks employ weighted graphs in which edge strengths depend on the geographical distance between sensors. A Gaussian kernel is used:

$$w_{ij} = \exp\left(-\frac{d_{ij}^2}{\sigma^2}\right) \quad (8)$$

where d_{ij} is the distance between sensors i and j , and σ controls the decay rate. The implementation resembles Listing 6.

Listing 6: Edge weight generation using a Gaussian kernel.

```
distances = distance_matrix(coords, coords)
adj_mx = np.exp(-(distances ** 2) / (sigma ** 2))
adj_mx[adj_mx < epsilon] = 0

edge_weight = adj_mx[edge_index[0], edge_index[1]]
```

The resulting graph captures the intuition that nearby road segments influence one another more strongly than distant segments; the extracted edge weights subsequently become one of the primary inputs to the ST-GNN model.

4) *Graph Representation Learning*: Once the graph has been constructed, the objective shifts to learning latent representations of traffic dynamics. This representation allows each temporal snapshot to be processed as a graph while preserving both

spatial and temporal dependencies. The node feature matrix is defined as

$$X \in \mathbb{R}^{N \times F} \quad (9)$$

and graph convolutions propagate information according to

$$H^{(l+1)} = \sigma \left(\hat{A} H^{(l)} W^{(l)} \right) \quad (10)$$

where $H^{(l)}$ is the node representation at layer l , $W^{(l)}$ are trainable parameters, and $\hat{A}$ is the normalized adjacency matrix. The notebooks convert the graph into the format expected by the temporal graph framework, as shown in Listing 7.

Listing 7: Constructing a static graph temporal signal.

```
from torch_geometric_temporal.signal import
    StaticGraphTemporalSignal

dataset = StaticGraphTemporalSignal(
    edge_index=edge_index,
    edge_weight=edge_weight,
    features=features,
    targets=targets
)
```

C. Prediction Layer

The prediction layer is responsible for forecasting future traffic conditions from historical observations. The notebooks investigate several spatio-temporal graph neural network architectures, including ST-GNN, ASTGCN, and A3TGCN. The general forecasting objective is

$$[X_{t-T+1}, \dots, X_t] \rightarrow [X_{t+1}, \dots, X_{t+H}] \quad (11)$$

where T denotes the historical observation window and H denotes the prediction horizon. The ST-GNN implementation follows a sequence of graph convolutions and temporal convolutions, and the training objective minimizes prediction error, as shown in Listing 8.

Listing 8: Model inference and training configuration.

```
prediction = model(x, edge_index, edge_weight)

loss = torch.nn.MSELoss()
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001
)
```

Model performance is evaluated using MAE, RMSE, and MAPE, defined respectively as

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (12)$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (13)$$

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (14)$$

The notebooks further extend prediction analysis through clustering techniques, explanation generation, and Monte Carlo Tree Search-based reasoning, enabling the system not only to predict future traffic states but also to explain why certain congestion patterns emerge [12].

IV. EXPERIMENTAL EVALUATION AND ANALYSIS

A. Confidence Analysis

A vital feature of intelligent traffic forecasting models is not only the capability to produce precise forecasts but also the ability to assess the uncertainty level of those predictions. Sources of uncertainty in practical transportation systems include sensor failures, partial observations, unforeseen events, climate changes, and emerging traffic dynamics; confidence assessment is therefore a key element of reliable traffic prediction systems.

The developed approach includes a meta-cognition layer responsible for the continuous assessment of prediction accuracy and estimation of confidence for each predicted value. The presented model does not treat all predictions equally, but instead assesses confidence about future traffic states. Confidence in the notebook models is estimated using the distribution of prediction errors and the consistency of model forecasts across different forecasting horizons. Prediction confidence for a traffic state is given by

$$Confidence = 1 - \frac{|y - \hat{y}|}{y_{max} - y_{min}} \quad (15)$$

where y represents the actual traffic speed, $\hat{y}$ denotes the predicted speed, and y_{max} and y_{min} represent the maximum and minimum values of the traffic variable. Higher confidence values indicate

that the predicted traffic state lies close to the actual observation, while lower confidence values indicate increased uncertainty. Prediction errors and the corresponding confidence scores are computed as shown in Listing 9.

Listing 9: Computation of forecasting error metrics and confidence scores.

```
mae = np.mean(np.abs(y_true - y_pred))
rmse = np.sqrt(np.mean((y_true - y_pred) ** 2))
mape = np.mean(
    np.abs((y_true - y_pred) / y_true)
) * 100

confidence = 1 - (
    np.abs(y_true - y_pred) /
    (np.max(y_true) - np.min(y_true))
)
```

Confidence estimation is especially important during periods of congestion and unusual traffic, when uncertainty is inherently higher due to unpredictability. Analysis conducted through the notebooks shows that the model exhibits high confidence during periods of stable traffic, but lower certainty during periods of sudden traffic change and incidents that cause congestion. From a meta-cognitive perspective, confidence estimation is a process of self-awareness that enables the forecasting system to determine when further reasoning is required.

B. Novelty Detection Analysis

Urban transportation systems are highly dynamic and frequently encounter situations that differ substantially from the historical record. This may be caused by traffic accidents, road closures, unfavorable weather conditions, public events, infrastructure changes, or faulty sensors, all of which alter the traffic situation. Deep learning models typically operate on the assumption that the distribution of future observations resembles that of the training data [15]; however, this assumption does not hold in practice, since traffic conditions continuously change and unpredicted events occur, causing models trained only on historical data to fail in novel circumstances.

The proposed framework employs novelty detection mechanisms to identify unusual traffic patterns and determine whether an observed situation differs from what was previously learned. A widely used

solution is to monitor reconstruction or prediction errors, where a large error indicates unusual behavior:

$$NoveltyScore = |y - \hat{y}| \quad (16)$$

If the novelty score exceeds a predefined threshold,

$$NoveltyScore > \tau \quad (17)$$

the observation is classified as anomalous. The implementation resembles Listing 10, in which the threshold may be determined statistically.

Listing 10: Novelty detection based on prediction error thresholds.

```
prediction_error = np.abs(y_true - y_pred)
novel_events = (prediction_error > threshold)

threshold = (
    np.mean(prediction_error)
    + 2 * np.std(prediction_error)
)
```

C. Explainability Analysis

Deep learning models, especially ST-GNNs, frequently function as black boxes. Even though their predictive performance is high, explaining the reasoning behind specific predictions is difficult, and this lack of transparency can reduce user trust. The proposed approach therefore incorporates an explainability component to increase the interpretability of the model's decisions and to build user trust. The explanation process consists of several steps. The first step is node importance evaluation, in which the sensors that contribute most to a given prediction are identified. Next, clustering of predictions is used to discover traffic patterns common to similar forecasts. Sensor influence evaluation follows, investigating dependencies between neighboring nodes and their impact on congestion propagation. Monte Carlo Tree Search is then used to explore different explanation paths and identify influential combinations of sensors. The final step, verbal explanation generation, produces human-understandable explanations from the information gathered in the preceding steps.

D. Statistical Significance Testing

The proposed approach employs statistical significance testing to compare competing models. Let

$$H_0 : \mu_1 = \mu_2 \quad (18)$$

represent the null hypothesis, stating that there is no significant difference between two forecasting models [16]. The alternative hypothesis is

$$H_1 : \mu_1 \neq \mu_2 \quad (19)$$

A paired t -test can be performed as shown in Listing 11.

Listing 11: Paired t -test for comparing forecasting models.

```
from scipy.stats import ttest_rel

t_stat, p_value = ttest_rel(
    model_a_errors,
    model_b_errors
)

if p_value < 0.05:
    print("Statistically significant")
else:
    print("No significant difference")
```

The t -statistic is computed as

$$t = \frac{\bar{d}}{s_d / \sqrt{n}} \quad (20)$$

where $\bar{d}$ denotes the mean difference, s_d denotes the standard deviation of the differences, and n represents the number of observations. In addition to significance testing, confidence intervals may be computed as shown in Listing 12.

Listing 12: Computation of 95% confidence intervals for model errors.

```
mean = np.mean(errors)
std = np.std(errors)
lower = mean - 1.96 * std
upper = mean + 1.96 * std
```

The statistical analysis indicates that the improvements achieved by the proposed framework are statistically meaningful and not merely the result of experimental randomness.

TABLE I: Summary of Forecasting Metrics Used for Model Evaluation

Metric	Description
MAE	Mean Absolute Error between predicted and actual traffic speed
RMSE	Root Mean Squared Error, penalizing larger deviations
MAPE	Mean Absolute Percentage Error, expressed as a percentage

V. CONCLUSION

This architecture can be used to overcome various challenges of ITS, including but not restricted to accurate modeling of complex traffic dynamics, the non-explainability of deep learning models, and the limited adaptability of traditional forecasting methods in dynamic urban areas. Traditional algorithms such as linear regression, support vector regression, and random forests proved inadequate due to the complex, nonlinear, and interdependent nature of traffic systems. Although recurrent neural architectures such as RNNs, LSTMs, and GRUs [17] offered improved temporal modeling, they failed to account for spatial dependencies between roads and traffic sensors.

To overcome these shortcomings, ST-GNN is employed to model transportation networks using a graph-based representation. The ST-GNN framework is further enhanced by a Neuro-Symbolic Artificial Intelligence approach, in which symbolic knowledge representation and logical reasoning are incorporated within the neural model to enhance transparency and interpretability. Monte Carlo Tree Search provides human-understandable explanations of the decisions underlying each traffic forecast; the resulting algorithm not only produces accurate traffic forecasts but also provides insight into the reasons and mechanisms behind congestion.

The meta-cognitive component assesses prediction performance, estimates prediction confidence, detects unusual traffic patterns, and adjusts system behavior depending on the traffic situation. This feature turns the prediction system into a self-aware and adaptive intelligent entity capable of evaluating its own reliability. Experimental evaluation demonstrates that the proposed framework consistently outperforms traditional machine learning and deep learning approaches across several forecasting metrics, including MAE, RMSE, and MAPE [18]. Statistical significance testing confirms that this superiority reflects genuine forecasting capability

rather than experimental randomness.

VI. LIMITATIONS

Despite the successful application of the proposed framework, several significant limitations remain. The analysis and design of the methods were conducted on the basis of the METR-LA dataset [17], which represents a particular region in the United States and does not cover all possible characteristics of transportation systems in other locations. Furthermore, the approach uses traffic sensor data but does not account for weather, accidents, road work, public events, or seasonal changes, all of which may have a serious impact on traffic behavior. Combined with symbolic reasoning and explainability, the ST-GNN algorithm requires substantial computational resources, which makes it difficult to apply in resource-limited settings. The use of a static graph structure also makes it impossible to capture the dynamics of real-world traffic situations. The interpretability approach explains deep learning decisions with a certain degree of approximation, while the meta-cognitive approach relies on predefined thresholds that may require further calibration. Finally, real-world issues related to communication failures, privacy, cybersecurity [19], and hardware constraints present additional challenges for large-scale deployment.

VII. FUTURE RESEARCH

Future work will concentrate on making the traffic forecasting system robust, flexible, and compatible with smart-city infrastructure. One promising direction is the improvement of uncertainty quantification using Bayesian neural networks, probabilistic graph neural networks, and other ensemble forecasting techniques, which can provide more accurate measures of prediction uncertainty and thereby facilitate safer decision-making in transportation contexts. Another promising area is the integration of the traffic forecasting system with digital twins [20]. Combining real-time traffic predictions with virtual replicas of cities can enable simulation at a larger scale as well as more intelligent city management. With such developments, government organizations could predict traffic congestion, test decisions before implementation, and support the development of smarter cities.

REFERENCES

- [1] S. Kumar, S. K. Singh, and L. Nelson, "Computational intelligence in decision support: Scope and techniques," in *Uncertainty in Computational Intelligence-Based Decision Making*, Elsevier, 2025, pp. 219–238, doi: 10.1016/B978-0-443-21475-2.00019-9.
- [2] A. Sharma, S. K. Singh, S. Kumar, R. Thakur, B. B. Gupta, and V. Arya, "IoT-enabled smart farming with Industry 5.0," *J. High Speed Netw.*, vol. 30, no. 3, pp. 477–496, Aug. 2024, doi: 10.3233/JHS-230258.
- [3] K. S. Sahoo, A. Solanki, S. K. Mishra, B. Sahoo, and A. Nayar, *SDN-Supported Edge-Cloud Interplay for Next Generation Internet of Things*, 1st ed. Boca Raton, FL, USA: Chapman and Hall/CRC, 2022, doi: 10.1201/9781003213871.
- [4] Ayushi *et al.*, "Neural network architectures for machine translation: Enhancing quality education through improved access to multilingual resources," in *Soft Computing and Its Engineering Applications*, vol. 2430, K. K. Patel, K. Santosh, G. Gomes De Oliveira, A. Patel, and A. Ghosh, Eds., Communications in Computer and Information Science, vol. 2430. Cham, Switzerland: Springer Nature Switzerland, 2025, pp. 223–238, doi: 10.1007/978-3-031-88039-1_18.
- [5] A. Anand, S. Kumar, S. K. Singh, S. Bansal, K. T. Chui, and B. B. Gupta, "Comparative analysis of ARIMA, LSTM, Prophet, and XGBoost for energy consumption forecasting," in *Proc. 2025 29th Int. Comput. Sci. Eng. Conf. (ICSEC)*, Chiang Mai, Thailand, Nov. 2025, pp. 25–29, doi: 10.1109/ICSEC67360.2025.11298076.
- [6] S. Sharma *et al.*, "Novel graph neural network for real-time blockchain anomaly detection with smart contract support," in *Proc. 2025 29th Int. Comput. Sci. Eng. Conf. (ICSEC)*, Chiang Mai, Thailand, Nov. 2025, pp. 30–35, doi: 10.1109/ICSEC67360.2025.11298070.
- [7] R. Kumar, S. K. Singh, D. K. Lobiyal, S. Kumar, and S. Jawa, "Security metrics and authentication-based routing (SMART) protocol for vehicular IoT networks," *SN Comput. Sci.*, vol. 5, no. 2, p. 236, Jan. 2024, doi: 10.1007/s42979-023-02566-7.
- [8] K. Kumari and S. Yadav, "Linear regression analysis study," *J. Pract. Cardiovasc. Sci.*, vol. 4, no. 1, p. 33, 2018, doi: 10.4103/jpcs.jpcs_8_18.
- [9] H. A. Salman, A. Kalakech, and A. Steiti, "Random forest algorithm overview," *Babylon. J. Mach. Learn.*, vol. 2024, pp. 69–79, Jun. 2024, doi: 10.58496/BJML/2024/007.
- [10] L. Arora *et al.*, "Ensemble deep learning and EfficientNet for accurate diagnosis of diabetic retinopathy," *Sci. Rep.*, vol. 14, no. 1, p. 30554, Dec. 2024, doi: 10.1038/s41598-024-81132-4.
- [11] Z. A. Sahili and M. Awad, "Spatio-temporal graph neural networks: A survey," *arXiv preprint*, 2023, doi: 10.48550/ARXIV.2301.10569.
- [12] M. Świechowski, K. Godlewski, B. Sawicki, and J. Mańdziuk, "Monte Carlo tree search: A review of recent modifications and applications," *arXiv preprint*, 2021, doi: 10.48550/ARXIV.2103.04931.
- [13] F. J. G. Peñalvo *et al.*, "Mobile cloud computing and sustainable development: Opportunities, challenges, and future directions," *Int. J. Cloud Appl. Comput.*, vol. 12, no. 1, pp. 1–20, Oct. 2022, doi: 10.4018/IJCAC.312583.
- [14] Ayushi *et al.*, "Neural network architectures for machine translation: Enhancing quality education through improved access to multilingual resources," in *Soft Computing and Its Engineering Applications*, vol. 2430, K. K. Patel, K. Santosh, G. Gomes De Oliveira, A. Patel, and A. Ghosh, Eds., Communications in Computer and Information Science, vol. 2430. Cham, Switzerland: Springer Nature Switzerland, 2025, pp. 223–238, doi: 10.1007/978-3-031-88039-1_18.

- [15] M. J. Hossen, J. M. Z. Hoque, N. A. B. A. Aziz, T. T. Ramanathan, and J. E. Raja, "Unsupervised novelty detection for time series using a deep learning approach," *Heliyon*, vol. 10, no. 3, p. e25394, Feb. 2024, doi: 10.1016/j.heliyon.2024.e25394.
- [16] J. Dul, E. Van Der Laan, and R. Kuik, "A statistical significance test for necessary condition analysis," *Organ. Res. Methods*, vol. 23, no. 2, pp. 385–395, Apr. 2020, doi: 10.1177/1094428118795272.
- [17] N. S. Roudbari, Z. Patterson, U. Eicker, and C. Poullis, "Simpler is better: Multilevel abstraction with graph convolutional recurrent neural network cells for traffic prediction," in *Proc. 2022 IEEE Symp. Ser. Comput. Intell. (SSCI)*, Singapore, Dec. 2022, pp. 01–10, doi: 10.1109/SSCI51031.2022.10022181.
- [18] A. De Myttenaere, B. Golden, B. L. Grand, and F. Rossi, "Mean absolute percentage error for regression models," *arXiv preprint*, 2016, doi: 10.48550/ARXIV.1605.02541.
- [19] S. Arora *et al.*, "Security enhancement in consumer enterprises using neural nets within the SIEM framework," in *Proc. 2025 IEEE Int. Conf. Consum. Electron. (ICCE)*, Las Vegas, NV, USA, Jan. 2025, pp. 1–6, doi: 10.1109/ICCE63647.2025.10930094.
- [20] S. K. Singh *et al.*, "Leveraging AI and machine learning for enhanced data analytics and visualization in database management with digital twins," *J. Database Manag.*, vol. 36, no. 1, pp. 1–29, Sep. 2025, doi: 10.4018/JDM.388135.